11-768 HW2: Evaluating a Data-Visualization Agent

Yuntian Shen

1 Part 1: Examine Seed Agent Runs

1.1 Comparison of Ground-Truth and Validator Labels

Summarize how the baseline validator's predictions compare with the provided human labels on the seed runs. Report all four per-family MCC scores and macro-MCC, and describe where the disagreements concentrate.

Baseline results:

Error family	MCC	TP	TN	FP	FN	Support (+/−)
`execution_failure`	−0.0022	4	65	33	8	12/98
`wrong_data`	0.0000	0	73	0	25	25/73
`wrong_chart`	0.1096	3	27	0	68	71/27
`hard_to_read`	0.0000	0	59	0	39	39/59

Macro-MCC is 0.0268.

The baseline missed every data and readability error and most chart errors. It also falsely reported execution failure in 33 runs.

1.2 Validator Error Type 1

Describe the first recurring type of validator error. Cite the run IDs of at least two runs that exhibit it, and explain why the baseline validator failed on them. Ground the analysis in specific evidence from the task, trajectory, input data, or figure.

The baseline incorrectly reports execution failure for runs like `004` and `022`. Both have final figures, and neither is labeled as an execution failure in the human annotations. The problem is that the baseline asks whether the agent failed, then treats any response that starts with YES as a failure. In these two runs, the saved explanations actually say the agent succeeded. The code does not check this contradiction. Once it reports execution failure, it stops and does not check for data, chart, or readability errors. This also causes it to miss the data and chart errors in run `004`.

1.3 Validator Error Type 2

Describe the second recurring type of validator error. Cite the run IDs of at least two runs that exhibit it, and explain why the baseline validator failed on them. Ground the analysis in specific evidence from the task, trajectory, input data, or figure.

The baseline missed data errors in runs like `016` and `028`. In `016`, the task says to remove browser versions with a share below 0.5%. The final code does not remove them. For example, IE 9.0 still appears in the chart even though its share is only 0.44%. In `028`, the blue line should

connect the 2015 data points. The final code sets `highlight = 2014`, so it connects the wrong year's data instead. The baseline reports no errors in either run. Both agents wrote earlier versions of the code with the correct conditions, but their final versions changed those conditions. The validator may have checked the earlier code and overlooked the code that actually produced the figures.

1.4 Proposed Validator Changes

For each error type above, propose a targeted change to the validator that might catch that class of error.

First, I would check whether the final image exists and can be opened. I would then ask the model whether the agent produced a valid final figure and why. The model should judge the final result, not an earlier crash. A second check would compare its decision with its explanation. If they disagree, I would ask the model to review the evidence again before accepting the result. This would replace the current rule that treats any YES answer as a failure.

Second, I would ask the validator to extract each required data filter, selection, and transformation. It would compare these requirements with the input data, final executed code, and image. The check should focus on the final version, so correct code from an earlier attempt cannot hide a later mistake.

2 Part 2: Improve Your Validator

2.1 Implemented Changes

Describe the two changes implemented from Part 1. Distinguish deterministic checks from model-based checks, where applicable, and explain how each change uses the task, trajectory, input data, or rendered figure.

First, I replaced the YES-prefix rule with a check of the final image. Missing or unreadable image files are marked as execution failures locally. For files that open successfully, the model returns a status and explanation in JSON, then a second call reviews them against the image before the code accepts the decision. The prompt makes clear that incorrect data or design does not count as execution failure if the agent produced a chart.

Second, I changed the data and chart checks to compare each task requirement with the input data, final executed code, and figure. To help the model focus on the final version, I omit earlier attempts when the tool records identify a complete script rewrite followed by a successful run. I keep all steps from that rewrite onward, or the full history if no such rewrite is identified. The model returns evidence and a status for each requirement, and the code reports only violations, not uncertain judgments.

The readability check remains unchanged from the baseline.

2.2 Seed-Set Performance vs. Baseline

Report the modified validator's performance on the seed runs alongside the baseline using all four per-family MCC scores and macro-MCC.

	Baseline	Modified validator				
Error family	MCC	MCC	TP	TN	FP	FN
`execution_failure`	−0.0022	0.8806	12	95	3	0
`wrong_data`	0.0000	0.2459	9	63	10	16
`wrong_chart`	0.1096	0.5014	39	27	0	32
`hard_to_read`	0.0000	−0.2424	0	51	8	39
Macro-MCC	0.0268	0.3464	–	–	–	–

2.3 Qualitative Analysis of Targeted Error Types

For each of the two error types surfaced in Part 1, discuss whether the corresponding change actually mitigated it, with reference to specific runs. Also describe any regressions.

The first change substantially reduced execution failures. False positives reduce from 33 to 3, and false negatives decrease from 8 to 0. Runs `004` and `022` are no longer marked as execution failures. However, `004` now receives no errors at all, so its data and chart errors are still not detected. Fixing the execution check did not guarantee that the later checks would work.

The second change helped only partly. The validator now detects the missing 0.5% filter in `016`. In `028`, its explanation identifies `highlight = 2014`, but it reports the wrong year as `wrong_chart`. The requested data is from 2015, so this is still a false negative for `wrong_data`. It found the relevant code but assigned the wrong family. This suggests that focusing on the final code helps, but does not resolve category confusion.

Run `013` is a false positive for `wrong_data`. The validator reports the rainbow colormap and x-axis limits of $[-5, 25]$ instead of $[0, 20]$ as both data and chart errors. These are chart-design errors under the assignment definitions. Together with `028`, this suggests a need for clearer category checks. Overall, data true positives rose from 0 to 9, but data false positives also rose from 0 to 10.

Readability became worse despite no changes to its check. It still misses all 39 positive cases and now adds 8 false positives. Fewer execution-failure predictions allow more runs to reach this check, so unchanged code does not imply unchanged results. The readability check remains a major weakness and we need to change the baseline.

3 Part 3: Design New Evaluation Tasks

3.1 Anticipated Generalization Weaknesses

Identify two classes of visualization task that your current validator may have trouble generalizing to, and explain why.

The first class is filtered aggregation. These tasks require the agent to select the right records before it calculates totals or rates. My validator asks the model to check the final code, but it does not independently calculate the expected values. It may accept a plausible chart even if the calculation is wrong.

The second class is crowded layouts with long labels or several panels. The current readability check missed all 39 positive seed cases, so I expect it may also miss clipped text, overlapping labels, or legends that cover data in new tasks. These tasks should ask for readable figures and leave enough space for a correct solution.

3.2 Task Design

Describe the five visualization tasks, including at least two from each weakness class. For each task, describe its inputs and data source, expected behavior, and why it is a useful stress test for the validator.

All five tasks use small synthetic datasets. Each task includes its data in the prompt, so it doesn't need a separate input file.

`channel-click-rates` tests the first weakness of filtered aggregation. The agent must keep only the final campaign records from 2025 and calculate each channel's rate from total clicks and impressions. The correct order is Email (12%), Search (9%), and Social (3%). An average of the individual rates gives the wrong answer because the records have different numbers of impressions. This tests whether the validator checks the calculation behind the bars.

`completed-order-revenue` also tests filtered aggregation. The agent must exclude cancelled orders, multiply quantity by unit price for each remaining order, and add the revenue by product. The bars should show Folder ($50), Notebook ($130), and Pen ($80), in that order. This tests whether the validator catches an incorrect filter or a calculation that ignores quantity.

`weekday-mode-shares` also tests filtered aggregation. The agent must only keep weekday trips from Central and draw a pie chart of the travel modes. The correct shares are Bus (60%), Bicycle (30%), and Walking (10%). Weekend trips and trips from West should not affect either the counts or the total used to calculate the percentages. This tests whether the validator checks which records were included in both parts of the calculation.

`course-enrollment-labels` tests the second weakness of crowded layouts. The agent must draw six bars with full course names and enrollment numbers. The long names can overlap or get cut off unless the agent adjusts the layout. Wrapping or rotating the names is allowed. This tests whether the validator can spot unreadable labels even when the data and chart type are correct.

`service-response-panels` also tests crowded layouts. The agent must draw two line charts for response times before and after an update, with three services in each panel. Each panel needs a legend with the full service names. This tests whether the validator can notice legends that cover lines or text, or labels that overlap across panels. The data requires no calculation, so the main challenge is to keep the figure readable.

3.3 Agent Runs and Labeling

Describe the agents run on the new tasks, the runs generated, and how you assigned human labels using the four fixed error families. Explain any ambiguous labeling decisions.

Out of 15 runs, 3 Qwen runs did not deliver the required `figure.png`, and its travel task only save a title on a blank canvas. These four runs are labeled only as `execution_failure`. Its course chart has the correct bar values, but the course names are cut off, the value labels are missing, and the figure size is wrong. Hence is is `wrong_chart` and `hard_to_read`.

Ministral and GLM correctly calculate the click rates, completed-order revenues, and weekday travel shares. Both also produce readable course charts. The main remaining problem is the Ministral service chart, where the two legends overlap and extend beyond the image edges. This is labeled `hard_to_read`. On these tasks, Ministral and GLM handled the data calculations well, but a multi-panel layout still cause a readability problem.

For ambiguous cases, I accept cropped blank margins when the code set the requested figure size and DPI. I also accept the black percentage text on the Ministral pie chart because it is readable to me despite its borderline contrast. Earlier crashes were not labeled as execution failures if the agent later produced a valid chart.

3.4 Validator Performance on New Tasks

Report all four per-family MCC scores and macro-MCC on the new agent runs and qualitatively analyze important successes, failures, and uncertain labels.

Error family	MCC	TP	TN	FP	FN	Support (+/−)
`execution_failure`	1.0000	4	11	0	0	4/11
`wrong_data`	0.0000	0	11	0	0	0/11
`wrong_chart`	1.0000	1	10	0	0	1/10
`hard_to_read`	−0.2222	0	7	2	2	2/9
Macro-MCC	0.4444	–	–	–	–	–

The validator correctly identifies all four execution failures, including the blank Qwen pie chart. It also catches the missing value labels and incorrect figure size in the Qwen course chart, with no false positives for `wrong_chart`. Both categories have MCC 1.0, but there is only one positive chart-error case, so this is a limited test of chart checking.

Readability is much less reliable. In `service-response-panels`, the Ministral legends visibly overlap and are clipped but the validator reports no errors. It also misses the readability label for the Qwen course chart, even though its chart-error explanation notices the clipped course names. Both `weekday-mode-shares` charts from GLM and Ministral are false positives. The validator claims that labels are cut off or hidden, but the images do not show those problems. It needs to check its visual claims more carefully and assign readability problems to the correct family.

No valid final chart has a data error, so the `wrong_data` MCC is 0 by convention. The validator makes no false data-error predictions, but this set does not test whether it can catch actual data errors.

3.5 Harbor Packaging

State that every authored task scores 1.0 against `-a oracle`; you may also report the recommended `-a nop` runs. Describe both plausible-but-wrong mutant solutions and report each reward. For the mutant scoring 0, explain what it gets wrong and name the assertion that rejects it. For the mutant scoring 1.0, explain what it gets wrong and why the verifier accepts it. Then, for each of the four error families, say whether a deterministic Harbor verifier for your task could decide it, and why or why not. Finally: your validator is graded on private runs for tasks you never authored. Explain why a per-task verifier cannot serve as that grading mechanism.

I packaged all five tasks with reference solutions and deterministic verifiers. All five tasks received reward 1.0 in Harbor with `-a oracle`. The provided example received 1.0 with `-a oracle` and 0 with `-a nop`. Its `illegible` mutant also received 1.0. These runs used Colima as the Docker backend with the task's no-network setting. Both of my mutants were also tested through Harbor.

The `unweighted-rates` mutant averages the individual click-through rates instead of dividing total clicks by total impressions. It received reward 0. The `math.isclose` assertion in `test_click_through_rates` rejected its Email rate of 20%, which should be 12%.

The `tiny-text` mutant uses the correct values but sets the font size to 2pt. It received reward 1.0 because the verifier checks the numeric output and basic image validity, but does not check whether the text is readable.

For `execution_failure`, the checks catch missing images, invalid PNGs, and simple blank canvases. They cannot reliably reject an unfinished chart that contains enough text or other marks to pass the image checks.

For `wrong_data`, the verifier compares `plotted_values.json` with known answers. This catches incorrect declared values, but does not prove that the image actually shows those values.

For `wrong_chart`, additional checks could inspect the plotting objects for chart type, series order, and labels. My current verifier does not inspect those properties, so correct numeric output can pass with an incorrect chart.

For `hard_to_read`, rules could flag small fonts or overlapping text, but basic file and numeric checks cannot decide readability. The accepted `tiny-text` mutant demonstrates this limitation.

These verifiers depend on the known answers and output formats of my five tasks. They cannot grade unseen private tasks without new checks and expected values for each task. The validator instead has to interpret each task and judge the agent's output from the available evidence.

4 Part 4: Iterate on Your Design

4.1 Additional Improvement Motivated by Part 3

Describe at least one change to the validator motivated by the Part 3 results. Identify the observed failure that motivated it and explain why you expected the change to help.

In Part 3, the readability check missed clipped course names and overlapping service legends, but incorrectly flagged two readable pie charts. I replaced its general YES/NO question with six checks from the assignment's readability definition. The model now examines the final image and task without the trajectory, so claims in earlier code do not substitute for visual evidence. Each check returns specific evidence and a status of clear, problem, or uncertain. Only a problem produces a readability error.

The prompt asks the model to locate each defect and distinguishes actual clipping or obstruction from text near an edge or inside a chart. I also clarify that partially clipped or overlapping text belongs in the readability check, even when the task explicitly asks for readable labels. These changes are intended to improve coverage and reduce unsupported visual claims.

4.2 Final Evaluation on Seed and New Tasks

Evaluate the final validator on both the seed runs and self-authored runs. Report all four per-family MCC scores and macro-MCC before and after the change on each subset, including any regressions.

Seed runs						
Error family	MCC	TP	TN	FP	FN	Support (+/−)
`execution_failure`	0.8482	12	94	4	0	12/98
`wrong_data`	0.2665	9	64	9	16	25/73
`wrong_chart`	0.3151	38	22	5	33	71/27
`hard_to_read`	0.0921	8	51	8	31	39/59
Macro-MCC	0.3805	–	–	–	–	–

Authored runs						
Error family	MCC	TP	TN	FP	FN	Support (+/−)
`execution_failure`	1.0000	4	11	0	0	4/11
`wrong_data`	0.0000	0	11	0	0	0/11
`wrong_chart`	0.6708	1	9	1	0	1/10
`hard_to_read`	0.6708	1	9	0	1	2/9
Macro-MCC	0.5854	–	–	–	–	–

4.3 Qualitative Analysis of Self-Authored Tasks

Discuss whether the change improved performance on your self-authored tasks, with reference to specific runs.

On the authored tasks, macro-MCC increases from 0.4444 to 0.5854. The validator now catches the Ministral service chart's overlapping legends and no longer flags the GLM and Ministral travel pie charts as unreadable. Readability false positives drop from 2 to 0, and false negatives drop from 2 to 1.

On seeds, macro-MCC increases from 0.3464 to 0.3805. Readability detects 8 true errors instead of 0, but chart checking adds 5 false positives and execution checking adds one. Data checking removes one false positive. These regressions limit the overall improvement.

The Ministral course chart is a new `wrong_chart` false positive. The validator demands a legend and colorbar, although neither is requested. It appears to mistake items in the general checklist for requirements of this task.

The Qwen course chart remains a `hard_to_read` false negative. Its code sets `rotation=45` and saves the figure without adjusting the layout. The course names are visibly cut off at the bottom, but the validator reports only chart-design errors. It may recognize the rotated labels without checking whether their full text fits inside the image.

4.4 Further Changes

Report any additional validator changes or tasks beyond the required ones. For each, explain its motivation and how successful you judged it to be, qualitatively and, for quantitative comparisons, using all four per-family MCC scores and macro-MCC.

NA